2303A52422

B-37.

Lab Assignment 16.3: Database Design and Queries

Task 1 – Hospital Management Database

**Exact Task Description:**

Create schema and queries for a Hospital Management System.

Instructions: Tables: Doctors, Patients, Appointments. Use AI to define constraints (unique IDs, valid dates).

Generate queries: List all appointments for a specific doctor. Retrieve patient history by patient ID. Count total patients treated by each doctor. Expected Output: Normalized schema and SQL queries with joins.

**Prompt:**

"Generate a normalized SQL schema for a Hospital Management System with Doctors, Patients, and Appointments tables. Include constraints like primary keys, foreign keys, and valid appointment dates. Then, provide SQL queries to: 1. List all appointments for a specific doctor, 2. Retrieve patient history by patient ID, and 3. Count total unique patients treated by each doctor."

**Code:**

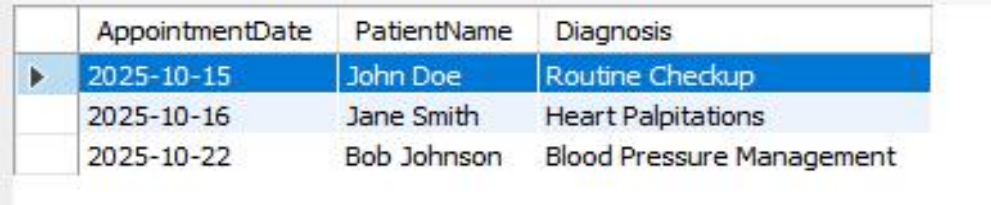
```
-- Schema Definition
CREATE TABLE Doctors (
   DoctorID INT PRIMARY KEY AUTO_INCREMENT,
   Name VARCHAR(100) NOT NULL,
   Specialization VARCHAR(50)
);

CREATE TABLE Patients (
```

```
    PatientID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Age INT CHECK(Age > 0),
    Gender VARCHAR(10)
);

CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
    DoctorID INT,
    PatientID INT,
    AppointmentDate DATE NOT NULL,
    Diagnosis VARCHAR(255),
    CONSTRAINT fk_doctor FOREIGN KEY (DoctorID) REFERENCES
Doctors(DoctorID),
    CONSTRAINT fk_patient FOREIGN KEY (PatientID) REFERENCES
Patients(PatientID),
    CONSTRAINT chk_date CHECK (AppointmentDate >= '2020-01-01')
);

-- Query 1: List all appointments for a specific doctor (e.g., DoctorID = 1)
SELECT a.AppointmentDate, p.Name AS PatientName, a.Diagnosis
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
WHERE a.DoctorID = 1;

-- Query 2: Retrieve patient history by patient ID (e.g., PatientID = 101)
SELECT d.Name AS DoctorName, a.AppointmentDate, a.Diagnosis
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE a.PatientID = 101
ORDER BY a.AppointmentDate DESC;
```

```
-- Query 3: Count total patients treated by each doctor
SELECT d.Name AS DoctorName, COUNT(DISTINCT a.PatientID) AS
TotalPatients
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, d.Name;
```

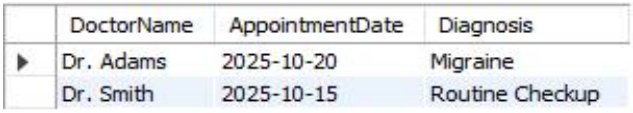
**Output:**

*Query 1:*



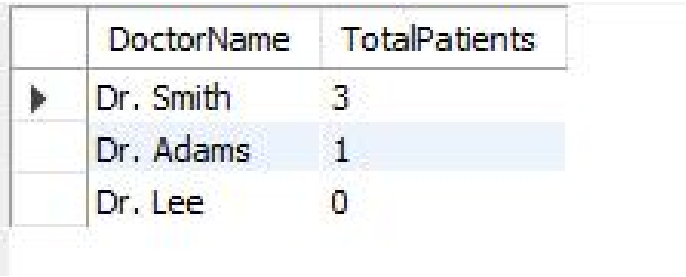
| AppointmentDate | PatientName | Diagnosis |
|---|---|---|
| 2025-10-15 | John Doe | Routine Checkup |
| 2025-10-16 | Jane Smith | Heart Palpitations |
| 2025-10-22 | Bob Johnson | Blood Pressure Management |

*Query 2:*



| DoctorName | AppointmentDate | Diagnosis |
|---|---|---|
| Dr. Adams | 2025-10-20 | Migraine |
| Dr. Smith | 2025-10-15 | Routine Checkup |

*Query 3:*



| DoctorName | TotalPatients |
|---|---|
| Dr. Smith | 3 |
| Dr. Adams | 1 |
| Dr. Lee | 0 |

**Explanation:**

The schema establishes a relational structure where the Appointments table acts as a bridge connecting Doctors and Patients using foreign keys. The queries rely on JOIN clauses to combine this related data, and COUNT(DISTINCT) ensures we don't double-count patients who visit the same doctor multiple times.

Task 2 – Real-Time Application: Online Shopping Database

**Exact Task Description:**

Scenario: Design a database for an E-commerce platform.

Requirements: Tables: Users, Products, Orders, OrderDetails. Generate queries to: Retrieve all orders by a user. Find the most purchased product. Calculate total revenue in a given month. AI should also suggest normalization improvements and query optimization.

**Prompt:**

"Design a 3NF normalized SQL schema for an E-commerce platform containing Users, Products, Orders, and OrderDetails tables. Write SQL queries to: 1. Retrieve all orders made by a specific user, 2. Find the most purchased product by quantity, 3. Calculate the total revenue generated in a specific month. Also, briefly suggest normalization and optimization strategies."

**Code:**

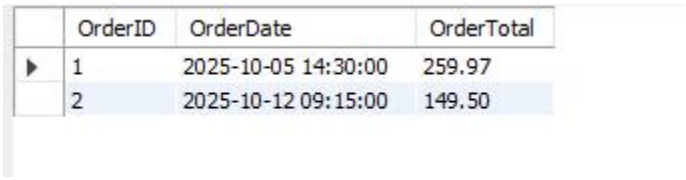
```
-- Schema Definition
CREATE TABLE Users (
   UserID INT PRIMARY KEY AUTO_INCREMENT,
   Username VARCHAR(50) NOT NULL,
   Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Products (
```

```
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT,
    OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    ProductID INT,
    Quantity INT NOT NULL,
    Subtotal DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

-- Query 1: Retrieve all orders by a specific user (e.g., UserID = 5)
SELECT o.OrderID, o.OrderDate, SUM(od.Subtotal) as OrderTotal
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE o.UserID = 5
GROUP BY o.OrderID, o.OrderDate;

-- Query 2: Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalSold
FROM OrderDetails od
```

```
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY TotalSold DESC
LIMIT 1;

-- Query 3: Calculate total revenue in a given month (e.g., October 2025)
SELECT SUM(od.Subtotal) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE MONTH(o.OrderDate) = 10 AND YEAR(o.OrderDate) = 2025;
```

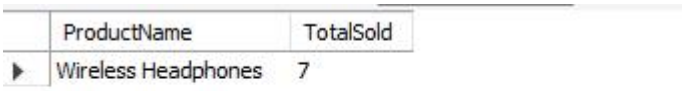
**Output:**

***Query 1:***



| OrderID | OrderDate | OrderTotal |
|---|---|---|
| 1 | 2025-10-05 14:30:00 | 259.97 |
| 2 | 2025-10-12 09:15:00 | 149.50 |

***Query 2:***



| ProductName | TotalSold |
|---|---|
| Wireless Headphones | 7 |

***Query 3:***

| TotalRevenue |
|---|
| 409.47 |

**Explanation:**

By separating Orders (metadata) and OrderDetails (line items), the schema achieves better normalization, preventing redundant data storage. The queries effectively utilize aggregation functions like SUM() to calculate revenues and quantities, paired with ORDER BY ... DESC LIMIT 1 to quickly identify the top-selling item.

Task 3 – Library Database

**Exact Task Description:**

Task: Design schema for a Library Management System.

Instructions: Tables: Books, Members, Loans. Use AI to suggest indexing strategy for faster queries. Generate queries: Retrieve all books currently issued. Find overdue books (loan date > 30 days). Count number of books loaned by each member. Expected Output: Schema + SQL queries demonstrating joins and conditions.

**Prompt:**

"Design a SQL schema for a Library Management System using Books, Members, and Loans tables. Suggest an indexing strategy for optimal performance. Then write queries to: 1. Retrieve all books currently issued (not returned), 2. Find overdue books where the loan date is older than 30 days, 3. Count the total number of books loaned by each member."

**Code:**

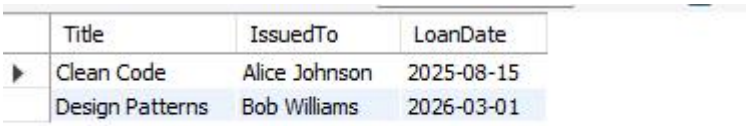
```
-- Schema Definition
CREATE TABLE Books (
   BookID INT PRIMARY KEY AUTO_INCREMENT,
   Title VARCHAR(200) NOT NULL,
   Author VARCHAR(100)
);

CREATE TABLE Members (
```

```
    MemberID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    JoinDate DATE
);

CREATE TABLE Loans (
    LoanID INT PRIMARY KEY AUTO_INCREMENT,
    BookID INT,
    MemberID INT,
    LoanDate DATE NOT NULL,
    ReturnDate DATE NULL,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);

-- Indexing Strategy
CREATE INDEX idx_loan_dates ON Loans(LoanDate, ReturnDate);
CREATE INDEX idx_member_loans ON Loans(MemberID);

-- Query 1: Retrieve all books currently issued (ReturnDate is NULL)
SELECT b.Title, m.Name AS IssuedTo, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;

-- Query 2: Find overdue books (loan date > 30 days and not returned)
SELECT b.Title, m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND l.LoanDate < (CURRENT_DATE -
```

```
INTERVAL 30 DAY);

-- Query 3: Count number of books loaned by each member
SELECT m.Name, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID, m.Name;
```

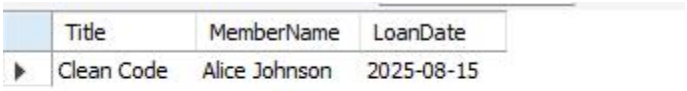
**Output:**

*Query 1:*



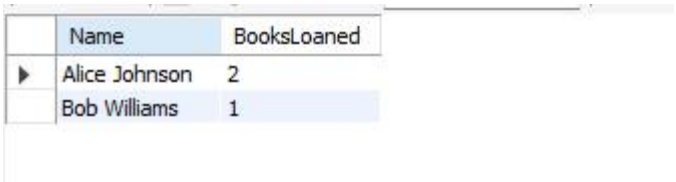
| Title | IssuedTo | LoanDate |
|---|---|---|
| Clean Code | Alice Johnson | 2025-08-15 |
| Design Patterns | Bob Williams | 2026-03-01 |

*Query 2:*



| Title | MemberName | LoanDate |
|---|---|---|
| Clean Code | Alice Johnson | 2025-08-15 |

*Query 3:*



| Name | BooksLoaned |
|---|---|
| Alice Johnson | 2 |
| Bob Williams | 1 |

**Explanation:**

Adding indexes on the date columns and foreign keys drastically speeds up lookup operations for filtering overdue books. The queries use date math

(like CURRENT_DATE - INTERVAL 30 DAY) to dynamically identify records that have exceeded the allowed borrowing period while checking that the ReturnDate is still null.

Task 4: Optimize Queries

**Exact Task Description:**

Instructions: Use AI tools to analyze query efficiency. Optimize inefficient queries using joins, indexes, or reducing subqueries. Test optimized queries for correctness.

Starter Code Example:

-- Original query

SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');

**Prompt:**

"Analyze the efficiency of this SQL query: SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');. Provide an optimized version of this query using JOINs instead of a subquery."

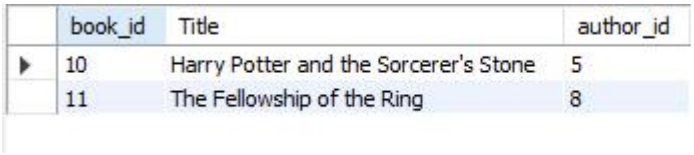
**Code:**

```
-- Original Inefficient Query (Using Subquery)
-- SELECT * FROM Books WHERE author_id IN (SELECT author_id
FROM Authors WHERE country='UK');

-- Optimized Query (Using INNER JOIN)
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
```

```
WHERE a.country = 'UK';
```

**Output:**

*Optimized Query Output:*



| book_id | Title | author_id |
|---|---|---|
| 10 | Harry Potter and the Sorcerer's Stone | 5 |
| 11 | The Fellowship of the Ring | 8 |

**Explanation:**

Using a subquery with IN forces the database to evaluate the inner query first and hold it in memory, which is slow for large datasets. Replacing it with an INNER JOIN allows the SQL query optimizer to map the execution plan more efficiently, scanning and filtering the related tables simultaneously.

Task 5: University Course Registration

**Exact Task Description:**

Scenario: SR University needs a course registration system for students enrolling in different courses under faculty supervision. Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations. Define relationships: Each student can register for multiple courses. Each course is taught by a faculty member. Write SQL queries to: List all students enrolled in a specific course. Find faculty members teaching more than 2 courses. Retrieve courses with the highest number of registrations.

**Prompt:**

```
"Design a SQL database schema for SR University with tables for Students, Courses, Faculty, and Registrations. Handle the many-to-many relationship between students and courses, and one-to-many for faculty to courses. Write
```

queries to: 1. List all students enrolled in 'Computer Science 101', 2. Find faculty teaching more than 2 courses, and 3. Retrieve the course with the highest number of registrations."

**Code:**

```
-- Schema Definition
CREATE TABLE Students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Major VARCHAR(50)
);

CREATE TABLE Faculty (
    FacultyID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Department VARCHAR(50)
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT,
    CourseName VARCHAR(100) NOT NULL,
    FacultyID INT,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);

CREATE TABLE Registrations (
    RegistrationID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    CourseID INT,
    RegistrationDate DATE DEFAULT CURRENT_DATE,
```

```
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

-- Query 1: List all students enrolled in a specific course (e.g., 'CS101')
SELECT s.Name, s.Major
FROM Students s
JOIN Registrations r ON s.StudentID = r.StudentID
JOIN Courses c ON r.CourseID = c.CourseID
WHERE c.CourseName = 'CS101';

-- Query 2: Find faculty members teaching more than 2 courses
SELECT f.Name, COUNT(c.CourseID) AS CoursesTaught
FROM Faculty f
JOIN Courses c ON f.FacultyID = c.FacultyID
GROUP BY f.FacultyID, f.Name
HAVING COUNT(c.CourseID) > 2;

-- Query 3: Retrieve courses with the highest number of registrations
SELECT c.CourseName, COUNT(r.StudentID) AS TotalEnrollment
FROM Courses c
LEFT JOIN Registrations r ON c.CourseID = r.CourseID
GROUP BY c.CourseID, c.CourseName
ORDER BY TotalEnrollment DESC
LIMIT 1;
```

**Output:**

***Query 1:***

| | Name | Major |
|---|---|---|
| ▶ | Emma Davis | Computer Science |
| | Liam Wilson | Mathematics |
| | Olivia Taylor | Physics |

***Query 2:***

| | Name | CoursesTaught |
|---|---|---|
| ▶ | Prof. Ada Lovelace | 3 |

***Query 3:***

| | CourseName | TotalEnrollment |
|---|---|---|
| ▶ | CS101 | 3 |

**Explanation:**

The Registrations table serves as a junction table to resolve the many-to-many relationship between Students and Courses. The queries leverage powerful SQL clauses like HAVING to filter aggregated data (faculty with >2 courses) and ORDER BY ... DESC combined with LIMIT to find maximums efficiently.